Learning Objectives

By the end of this session, you should be able to:

- Explain how expression trees are recursively translated into assembly instructions.
- Understand how register allocation and reuse work during code generation.
- Connect the expression parser, expression trees, and code generator to produce executable ULM programs.
- Extend the grammar and code generator to support arithmetic expressions and function calls.
- Apply the calling conventions developed in Session 14 to generate programs that return values and produce output.

Keeping Your Work

In this worksheet, we will continue working on our compiler project. All files should be created or modified directly in your `not-abc` repository.

Step 1 — From Expression Trees to Assembly Code (Preparation)

In Session 16, we extended our expression trees with additional node types. Now it is time to put them to work: we want to generate assembly code that evaluates an expression.
As usual, we start with a small test program. Create `xtest_expr_gen.abc` in your `not-abc` repository:

```
@ <stdio.hdr>
@ "expr.hdr"

fn main()
{
    local expr: -> Expr;

    expr = createIntegerExpr(UStrCreate("42"));

    printf("\t# load expression: ");
    printExprTree(expr);
    printf("\n");

    // loadExpr(expr);

    releaseExpr(expr);
}
```

Run `make` and execute `./xtest_expr_gen`. The program should compile and produce:

```
    # load expression: [42]
```

Notice that the output of `printExprTree()` is now an assembly comment. This will help us identify which expression the generated instructions belong to.

Next, we extend the interface of our expression-tree module. At the beginning of `expr.hdr`, include the header of our code generator:

```
@ "gen.hdr"
```

At the end of `expr.hdr`, add the declaration:

```
extern fn loadExpr(expr: ->Expr): GenReg;
```

The purpose of `loadExpr()` is to generate assembly instructions that evaluate the given expression and leave its result in a register.
Note the distinction: `loadExpr()` does not evaluate the expression itself. It generates code that will evaluate the expression when the generated assembly program is executed. The returned `GenReg` identifies the register in which that result will be available.

Task

Remove the comment from the following line in `xtest_expr_gen.abc`:

```
    loadExpr(expr);
```

Run `make` again. The test program should now compile, but the build should fail during linking because `loadExpr()` has been declared but not yet implemented.

Perfect! We now have a test program that uses our new interface, and the linker tells us exactly what is still missing.

We are ready to implement `loadExpr()` step by step.

© Michael C. Lehn | CC BY-SA 4.0

Step 2 — Generating Code for Arithmetic Expressions

We now have a test program that uses `loadExpr()`, but the implementation is still missing.
Of course, we already know that our expression trees can contain different kinds of nodes. Eventually, `loadExpr()` will have to handle all of them, just like `printExprTree()`. So rather than starting from scratch, we can reuse the design pattern of `printExprTree()`: inspect the node kind, distinguish between the different groups of expressions, and handle individual node kinds accordingly.
For now, however, we only need to support `EXPR_INTEGER`. And we already have a code generator function for loading constants into registers: `genConstant()`.
Add the following implementation to `expr.abc`:

```
fn loadExpr(expr: ->Expr): GenReg
{
    assert(expr);
    if (expr->kind >= EXPR_PRIMARY && expr->kind < EXPR_PRIMARY_END) {
        switch (expr->kind) {
            case EXPR_INTEGER: return genConstant(expr->primary.decimal);
            default: assert(0 && "primary node kind not handled");
        }
    } else {
        assert(0 && "case not handled");
    }
    return 0; // never reached
}
```

Notice that `genConstant()` already returns a `GenReg` identifying the register into which the constant will be loaded. We simply pass this register back to the caller of `loadExpr()`.

Tasks

1. **Testing integer constants.** Run `make` and execute `./xtest_expr_gen`. The generated assembly should look like this:
   ```
   # load expression: [42]
       .text
       loadz   42,     %4
   ```
2. **Testing binary expressions.** Now modify `xtest_expr_gen.abc` to construct a binary expression:
   ```
   expr = createBinaryExpr(EXPR_ADD,
                           createIntegerExpr(UStrCreate("20")),
                           createIntegerExpr(UStrCreate("22")));
   ```
   Run `make` and execute the test program. Compilation succeeds, but an assertion fails.
3. **Supporting binary expressions.** Extend `loadExpr()` as follows:
   ```
   fn loadExpr(expr: ->Expr): GenReg
   {
       assert(expr);
       if (expr->kind >= EXPR_PRIMARY && expr->kind < EXPR_PRIMARY_END) {
           switch (expr->kind) {
               case EXPR_INTEGER: return genConstant(expr->primary.decimal);
               default: assert(0 && "primary node kind not handled");
           }
       } else if (expr->kind >= EXPR_BINARY && expr->kind < EXPR_BINARY_END) {
           local left: GenReg = loadExpr(expr->binary.left);
           local right: GenReg = loadExpr(expr->binary.right);
           genReleaseReg(left);
           genReleaseReg(right);
           switch (expr->kind) {
               case EXPR_ADD: return genInstr(GEN_ADD, left, right);
               default: assert(0 && "binary node kind not handled");
           }
       } else {
           assert(0 && "case not handled");
       }
       return 0; // never reached
   }
   ```
   For a binary expression, we first recursively generate code that evaluates both child expressions and leaves their results in registers. We then release these registers so they can be reused. Finally, `genInstr()` generates the arithmetic instruction and returns the register containing its result. Run the test program again and inspect the generated assembly.

© Michael C. Lehn | CC BY-SA 4.0

More Expressions, More Tests?

We could now write a larger test program containing all the expression node kinds supported by our compiler. But there is an easier way: **we already have a parser that constructs expression trees for us!** Why build increasingly complicated expression trees by hand when we can simply parse them from source code?

Step 3 — Generating Code from Parsed Expressions (Preparation)

Copy `xtest_parseexpr.abc` to `xtest_parseexpr_gen.abc` and modify it as follows:

```
@ <stdio.hdr>
@ "lexer.hdr"
@ "parseexpr.hdr"

fn main()
{
    getToken();
    local expr: ->Expr = parseExpr();
    if (expr && token.kind == EOI) {
        printf("\t# load expression: ");
        printExprTree(expr);
        printf("\n");
        loadExpr(expr);
        releaseExpr(expr);
    }
    printf("\n");
}
```

Run `make` and test the program with a few arithmetic expressions that our code generator already supports. For example:

```
./xtest_parseexpr_gen
1+2+3
^D
```

This should produce:

```
# load expression: [ + [ + [1] [2]]  [3]]
    .text
    loadz   1,      %4
    loadz   2,      %5
    addq    %4,     %5,     %4
    loadz   3,      %5
    addq    %4,     %5,     %4
```

A Few Shell Tricks for Testing

Since Session 1, we have learned quite a few useful shell tricks. Let's put some of them to work.

Using a pipe. Instead of entering the expression interactively, we can use:

```
echo "1+2+3" | ./xtest_parseexpr_gen
```

The pipe (`|`) connects the standard output of the command `echo` to the standard input of `xtest_parseexpr_gen`. No need to type `Ctrl+D`.

Redirecting input. Create a file `test_expr_01.in` containing:

```
1 + 2 + 3
```

Then run:

```
./xtest_parseexpr_gen < test_expr_01.in
```

The `<` operator redirects standard input from a file.

Redirecting output and checking results. We can also save the generated assembly code:

```
./xtest_parseexpr_gen < test_expr_01.in > test_expr_01.out
```

The `>` operator redirects standard output to a file. Suppose another file, `test_expr_01.chk`, contains the expected output. We can compare the two files:

```
diff test_expr_01.chk test_expr_01.out
```

If `diff` produces no output, the files are identical. And just like that, we have the basic ingredients for automated testing: an input file, an expected output file, and a command to check whether our compiler produces the expected result.

© Michael C. Lehn | CC BY-SA 4.0

Step 4 — Supporting All Arithmetic Operators

Our parser currently supports the following grammar for expressions:

```
expr                = additive-expr
additive-expr       = multiplicative-expr { ("+" | "-") multiplicative-expr }
multiplicative-expr = factor { ("*" | "/" | "%") factor }
factor              = decimal-literal
                    | "(" expr ")"
```

In this step, we want to make sure that our code generator can handle every expression tree that our parser can produce.
Fortunately, there are not many cases to consider yet:

- **Decimal literals:** These produce nodes with `kind == EXPR_INTEGER`, which `loadExpr()` already handles.
- **Additive and multiplicative expressions:** These produce nodes belonging to the expression group `EXPR_BINARY`. So far, `loadExpr()` only handles `EXPR_ADD`. We still need to support `EXPR_SUB`, `EXPR_MUL`, `EXPR_DIV`, and `EXPR_MOD`.
- Parenthesized expressions: Parentheses affect how the expression tree is constructed, but do not require a separate node kind or special code generation.

Tasks

1. **Supporting subtraction.** Back in Session 10, we implemented code generation for both addition and subtraction. A quick look at `gen.hdr` reminds us of the relevant interface:

```
enum GenInstr
{
    GEN_ADD,
    GEN_SUB,
};

extern fn genInstr(instr: GenInstr, left: GenReg, right: GenReg): GenReg;
```

How these operations are implemented in `gen.abc` is not important here — hopefully, we tested them thoroughly back in Session 10!
We can therefore extend the switch statement in `loadExpr()` (in `expr.abc`) directly:

```
// ...
switch (expr->kind) {
    case EXPR_ADD: return genInstr(GEN_ADD, left, right);
    case EXPR_SUB: return genInstr(GEN_SUB, left, right);
    default: assert(0 && "binary node kind not handled");
}
// ...
```

Run `make` and test subtraction:

```
echo "2 - 3" | ./xtest_parseexpr_gen
```

Expected output:

```
    # load expression: [ - [2] [3]]
    .text
    loadz   2,   %4
    loadz   3,   %5
    subq    %5,  %4,  %4
```

Note: The instruction `subq %5, %4, %4` is correct! In our ULM assembly syntax, it means subtract the value in `%5` from the value in `%4` and store the result in `%4`.

2. **Supporting multiplication, division, and modulo.** Before we can handle these operators in `loadExpr()`, we need to extend our code generator. As you can see in the ULM Reference Manual for `hpc0.isa`, the instruction set provides `imulq`, `iquotq`, and `iremq` for these integer operations:
   - `imulq %1, %2, %3` multiplies the values in `%1` and `%2` and stores the result in `%3`.
   - `iquotq %1, %2, %3` divides the value in `%2` by the value in `%1` and stores the quotient in `%3`.
   - `iremq %1, %2, %3` divides the value in `%2` by the value in `%1` and stores the remainder in `%3`.

   Notice that, just as with `subq`, the right operand comes first in the instructions for division and modulo.
   Extend `GenInstr` in `gen.hdr` with `GEN_MUL`, `GEN_DIV`, and `GEN_MOD`. Then in `gen.abc` extend `genInstr()` such that it can generate the corresponding ULM assembly instructions.
   Test your implementation with `xtest_parseexpr_gen`.

© Michael C. Lehn | CC BY-SA 4.0

Step 5 — A Closer Look at Register Allocation

Now that our code generator supports all arithmetic operators, let's experiment with more complicated expressions.

You may notice something interesting: the generated code always leaves the final result in register `%4`, the register with the smallest ID that `getReg()` in `gen.abc` can allocate.

Is this just a coincidence?

Tasks

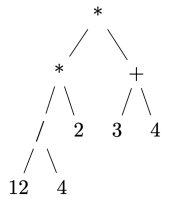
1. Try some more complicated expressions with `xtest_parseexpr_gen`. For example:

   ```
   echo "12 / 4 * 2 * (3 + 4)" | ./xtest_parseexpr_gen
   ```

   Try a few other expressions as well. Does the final result always end up in `%4`?
2. **Investigate why this actually is always the case.** Examine how `loadExpr()` recursively generates code and how `getReg()` and `genReleaseReg()` manage the available registers.

   It may help to visualize the expression tree, as we did in Worksheet 13. Our `Makefile` makes this particularly easy:

   (i) Run `make tree.tex`.
   (ii) When prompted, enter an expression such as `12 / 4 * 2 * (3 + 4)` and press `Ctrl+D` to signal end-of-file.
   (iii) This generates `tree.tex`. Compile it with `lualatex` to obtain a PDF showing the expression tree. The `Makefile` also prints the necessary instructions.

   Use the resulting tree to trace the recursive calls to `loadExpr()`. Keep track of which registers are allocated, released, and reused.

Step 6 — From Expression Trees to Executable Programs

So far, we have only inspected the assembly code generated for our expressions. Now let's take the next step: generate an executable program that actually computes the result!
Fortunately, we already did most of the necessary work on our code generator back in Session 14. We only need a small modification to get something quite remarkable.

Tasks

1. Generating a complete assembly program. Modify `xtest_parseexpr_gen.abc` as follows:

```
@ <stdio.hdr>
@ "lexer.hdr"
@ "parseexpr.hdr"

fn main()
{
    genInit();
    getToken();
    genFuncDefBegin("main");
    local expr: ->Expr = parseExpr();
    if (expr && token.kind == EOI) {
        printf("\t# load expression: ");
        printExprTree(expr);
        printf("\n");
        loadExpr(expr);
        releaseExpr(expr);
    }
    printf("\n");
    genFuncDefEnd();
}
```

   What has changed? We now use three functions from our code generator:
   - `genInit()` generates the startup code for our program.
   - `genFuncDefBegin("main")` generates the beginning of a function named main, including its prologue.
   - `genFuncDefEnd()` generates the function epilogue and return instruction.

© Michael C. Lehn | CC BY-SA 4.0

2. Run `make`, then generate and assemble a program:

   ```
   echo "12 / 4 * 2 * (3 + 4)" | ./xtest_parseexpr_gen > foo.s
   ulmas foo.s
   ```

   You can now use our ULM debugger to execute the program step by step:

   ```
   udb-tui a.out
   ```

   Inspect the generated instructions and follow how the expression is evaluated.
3. **Returning the result.** In the previous step, we observed that the result of an expression always ends up in register `%4`. Let's use this observation to make our generated main function return that value. Modify `genFuncDefEnd()` in `gen.abc`:

   ```
   fn genFuncDefEnd()
   {
       setSegment(TEXT);
       printf("\tmovq \t%%4, \t8(%%1)\n");
       printf("\tmovq \t(%%1), \t%%3\n");
       printf("\taddq \t8, \t%%1, \t%%1\n");
       printf("\tret \t%%3\n");
   }
   ```

   Only one line is new:

   ```
       printf("\tmovq \t%%4, \t8(%%1)\n");
   ```

   This stores the value of register `%4` in the return-value slot on the stack.

   This is a good opportunity to revisit the stack-frame diagrams from Worksheet 14. Make sure you understand why `8(%1)` is the correct location for the return value.
4. Run `make` again, then:

   ```
   echo "12 / 4 * 2 * (3 + 4)" | ./xtest_parseexpr_gen > foo.s
   ulmas foo.s
   ulm a.out
   echo $?
   ```

   The shell variable `$?` contains the exit status of the most recently executed command. So you should see the result `42`!

   Note: We have just built a compiler that translates arithmetic expressions into executable machine code! And thanks to the groundwork from Session 14, we needed surprisingly little new code to get here.

Step 7 — Extending the Grammar: Function Calls

So far, our parser only supports arithmetic expressions. We now want to extend it with function calls. You might expect us to introduce variables and identifiers first. But we will take a slightly different route: **we will support function calls before introducing variables.**

Why? Because we already have most of the necessary machinery! In Session 14, we extended our code generator to support function calls, and in Session 16, we introduced expression-tree nodes representing function calls and their arguments. Our extended grammar will be:

```
expr                = additive-expr
additive-expr       = multiplicative-expr { ("+" | "-") multiplicative-expr }
multiplicative-expr = factor { ("*" | "/" | "%") factor }
factor              = decimal-literal
                    | identifier “(“ [ expr { "," expr } ] ”)"
                    | "(" expr ")"
```

Notice that an identifier is only accepted as the name of a function being called. We do not yet support identifiers as expressions representing variables.
For example, the grammar now allows:

```
putchar(42)
putchar(40 + 2)
foo(1, 2 + 3, 4)
```

But not yet:

```
x + 1
```

Our goal is to extend the parser so that it constructs the EXPR_CALL nodes and argument lists introduced in Session 16.

© Michael C. Lehn | CC BY-SA 4.0

Tasks

1. When parsing function calls, we need to check whether certain tokens appear where expected. To avoid repeating the same checks, add the following helper function to `parseexpr.abc`:

```
fn expected(kind: TokenKind)
{
    if (token.kind != kind) {
        fprintf(stderr, "%u.%u: '%s' expected\n", token.pos.row, token.pos.col,
                tokenKindStr(kind));
        exit(1);
    }
}
```

   Also add its declaration to `parseexpr.hdr`:

```
extern fn expected(kind: TokenKind);
```

2. Parsing Function Calls. Now we can extend parseFactor() to handle the new grammar rule:

```
factor = decimal-literal
       | identifier "(" [ expr { "," expr } ] ")"
       | "(" expr ")"
```

   We already know how to parse decimal literals and parenthesized expressions. The new case is a function call. In `parseexpr.abc`, extend `parseFactor()` as follows:

```
fn parseFactor(): ->Expr
{
    if (token.kind == DECIMAL_LITERAL) {
        // as before
    } else if (token.kind == IDENTIFIER) {
        local ident: ->UStr = token.val;
        getToken();
        expected(LPAREN);
        getToken();
        local callExpr: ->Expr = createCallExpr(ident);
        local argExpr: ->Expr = parseExpr();
        if (argExpr) {
            addArgExpr(callExpr, argExpr);
            while (token.kind == COMMA) {
                getToken();
                argExpr = parseExpr();
                if (!argExpr) {
                    fprintf(stderr, "expression expected\n");
                    assert(0);
                }
                addArgExpr(callExpr, argExpr);
            }
        }
        expected(RPAREN);
        getToken();
        return callExpr;
    } else if (token.kind == LPAREN) {
        // as before
    } else {
        return nullptr;
    }
}
```

   There are two things worth looking at more closely:

   (i) **From grammar rules to control flow.** We already encountered this connection in Worksheets 13 and 15:
   - The optional part `[ expr { "," expr } ]` is handled by `if (argExpr)`.
   - The repetition `{ "," expr }` is handled by the `while` loop.
   - The required parentheses are checked using `expected()`.

   Notice how directly the structure of the grammar is reflected in the parser implementation.

   (ii) **Constructing the expression tree.** We introduced `createCallExpr()` and `addArgExpr()` in Session 16. Now we finally put them to use:
   - `createCallExpr(ident)` creates an `EXPR_CALL` node containing the function name.
   - `addArgExpr(callExpr, argExpr)` adds a parsed argument expression to the call's argument list.

© Michael C. Lehn | CC BY-SA 4.0

Run `make` and test the extended parser with expressions such as:

```
echo "putchar(42)" | ./xtest_parseexpr
echo "putchar(40 + 2)" | ./xtest_parseexpr
echo "foo(1, 2 + 3, 4)" | ./xtest_parseexpr
echo "getchar()" | ./xtest_parseexpr
echo "putchar(getchar())" | ./xtest_parseexpr
```

Inspect the generated expression trees. Do they reflect the grammar?

3. **Generating Code for Function Calls.** Our parser can now construct expression trees containing function calls. But our code generator does not yet know how to handle `EXPR_CALL` nodes. Try:

   ```
   echo "putchar(65)" | ./xtest_parseexpr_gen
   ```

   As expected, this triggers an assertion failure in `loadExpr()`.

   Fortunately, we already implemented the necessary code generator functions in Worksheet 14: `genCallBegin()`, `genCallAddArg()`, and `genCallEnd()`.

   Extend the switch statement for primary expressions in `loadExpr()` (`expr.abc`) as follows:

   ```
   // ...
   switch (expr->kind) {
       case EXPR_INTEGER: return genConstant(expr->primary.decimal);
       case EXPR_CALL:
           genCallBegin(expr->primary.call.fnName, expr->primary.call.argListLen);
           for (local n: ->Expr = expr->primary.call.argList; n; n = n->arg.next) {
               local argReg: GenReg = loadExpr(n->arg.expr);
               genReleaseReg(argReg);
               genCallAddArg(argReg, n->arg.pos);
           }
           return genCallEnd();
       default: assert(0 && "primary node kind not handled");
   }
   // ...
   ```

   This should look familiar from Worksheet 14:

   - `genCallBegin()` prepares the stack for the function call, including space for its arguments and return value.
   - For each argument, `loadExpr()` recursively generates code to evaluate the argument.
   - `genCallAddArg()` then generates code to store its value in the appropriate argument slot.
   - `genCallEnd()` generates the actual function call and returns the `GenReg` holding its return value.

   Notice that we use `n->arg.pos` to identify the argument position. This is important because, as we saw in Session 16, our argument list is stored in reverse order.

   Run `make` and try the test again:

   ```
   echo "putchar(65)" | ./xtest_parseexpr_gen
   ```

   Inspect the generated assembly. Can you recognize the calling convention introduced in Worksheet 14?

4. **Let's see it in action!** We can now generate, assemble, and execute a program that actually produces visible output:

   ```
   echo "putchar((15 + 5) * 3 + 5) + putchar(10)" | ./xtest_parseexpr_gen > foo.s
   ulmas foo.s
   ulm a.out
   ```

   The output should be: the letter `A` followed by a newline!

Final Remarks

Our grammar still lacks unary minus, although our expression trees already support it. We could easily add it now, but we'll postpone this until we introduce other unary operators, such as dereferencing and address-of.

We also took a shortcut by treating function calls as primary expressions with an identifier as the function name. This prevents us from supporting function pointers later without changing the grammar. For now, that's perfectly fine.

Most importantly, our generated programs can finally produce visible output instead of communicating with the outside world only through exit codes!

© Michael C. Lehn | CC BY-SA 4.0